

Chapter 22: Subjects

1. Overview

A plain RxJS `Observable` is **unicast** and typically **lazy**: every call to `.subscribe()` runs the producer function again, and each subscriber gets its own independent execution and its own private data stream. That model is perfect for things like HTTP calls or timers where you want fresh, isolated execution per consumer — but it is the wrong model for **shared state** and **event buses**, where you want one producer and many listeners hearing the *same* values at the *same* time.

A **Subject** solves this. A `Subject<T>` is simultaneously:

- An **Observable** — you can `.subscribe()` to it.
- An **Observer** — it has `.next()`, `.error()`, and `.complete()` methods, so *you* can push values into it imperatively from anywhere in your code (a click handler, an HTTP callback, a service method).

Because it is both, a Subject acts as a bridge between imperative, "push this value now" code and the declarative Observable world. It is also **multicast** by nature: every subscriber shares the same execution and receives the same values, emitted at the same physical time.

In Angular applications, Subjects (and their variants) are the backbone of:

- **Cross-component / cross-service communication** that doesn't naturally flow through `@Input()` / `@Output()` (siblings, unrelated components, deeply nested trees).
- **Shared state services** (`BehaviorSubject`-backed stores — the pattern that predates and still coexists with NgRx/signals).
- **Notification/event buses** (toast messages, "user logged out", "cart updated").
- **Caching the latest N emissions** for late subscribers (`ReplaySubject`).
- **Cleanup signals** (the classic `private destroy$ = new Subject<void>() + takeUntil(this.destroy$)` pattern).

With Angular's move toward signals, Subjects remain the natural "event" primitive (signals model *state*, not discrete events), and RxJS interop functions (`toSignal()`, `toObservable()`) let you bridge the two worlds cleanly. This chapter covers all four Subject variants in depth, their internal mechanics, and how they fit into modern Angular architecture.

2. Core Concepts

2.1 Unicast vs Multicast

Unicast (plain Observable):

```
const obs$ = new Observable<number>(subscriber => {
  console.log('Producer executed');
  subscriber.next(Math.random());
});

obs$.subscribe(v => console.log('A', v)); // "Producer executed", "A 0.123"
obs$.subscribe(v => console.log('B', v)); // "Producer executed", "B 0.987"
```

Each subscriber triggers the producer function independently — two different random numbers, two separate executions. This is unicast: one producer *per subscriber*.

Multicast (Subject):

```
const subject = new Subject<number>();
const obs$ = new Observable<number>(subscriber => {
  console.log('Producer executed');
  subscriber.next(Math.random());
});

obs$.subscribe(subject); // Subject subscribes as the single Observer
subject.subscribe(v => console.log('A', v));
subject.subscribe(v => console.log('B', v));
```

Here the producer runs once; the Subject fans that single execution out to both A and B, and they receive the *same* value. This is the essence of multicasting, and it's exactly what operators like `share()`, `shareReplay()`, and `connectable()` implement under the hood using Subjects.

2.2 Subject as both Observable and Observer

```
class Subject<T> extends Observable<T> implements SubscriptionLike {
  next(value: T): void { /* push to all current observers */ }
  error(err: any): void { /* push error, then teardown */ }
  complete(): void { /* mark completed, then teardown */ }
}
```

- As an **Observer**, it exposes `next/error/complete` so any code — a component method, an HTTP `.subscribe()` callback, a `setTimeout` — can feed it values.
- As an **Observable**, it exposes `subscribe()` so consumers can listen.

This dual nature is why you can do `someObservable$.subscribe(mySubject)` — a Subject *is* a valid Observer, so it can be passed directly as the subscribe argument to "tap into" another stream and re-broadcast it.

2.3 The Four Subject Types — Deep Comparison

Feature	Subject<T>	BehaviorSubject<T>	ReplaySubject<T>	AsyncSubject<T>
Requires initial value	No	Yes (constructor arg mandatory)	No	No
Current value accessible synchronously	No	Yes, via <code>.value</code> (or <code>.getValue()</code>)	No direct getter	No direct getter

Feature	<code>Subject<T></code>	<code>BehaviorSubject<T></code>	<code>ReplaySubject<T></code>	<code>AsyncSubject<T></code>
What a late subscriber immediately receives	Nothing (only future emissions)	The current/last value, replayed immediately	The last N buffered values (default: all, or bounded by <code>bufferSize</code>), replayed in order	Nothing until complete , then only the last emitted value
Buffer size	0 (no buffer)	1 (just the current value)	Configurable (<code>bufferSize</code> , default <code>Infinity</code>)	1 (only the final value, held until complete)
Time-based expiry	N/A	N/A	Optional <code>windowTime</code> param — old buffered values expire	N/A
Emits on subscribe if not yet completed	No	Yes (the current value)	Yes (buffered values)	No — waits for <code>complete()</code>
Emits after <code>.complete()</code> to new subscribers	No (subject is done; no replay)	Yes — replays the last value even after completion	Yes — replays buffer even after completion	Yes — this is its <i>entire</i> purpose
Typical use case	Event bus, notifications, one-off pub/sub	Shared/derived state with a "current value" (store pattern)	Caching last N results (e.g., last N API responses), replay-on-late-subscribe	Emitting a single final result only when a process finishes (rarely used in app code)
Errors on subscribe if not yet completed	No	No	No	No
Common Angular use	<code>destroy\$</code> , generic pub/sub services	<code>CartService</code> , <code>AuthService.currentUser\$</code> , form state stores	Undo/redo history, "last 5 log messages" panel, request caching	Rarely used directly; conceptually similar to a resolved <code>Promise</code>

Key exam-grade distinctions:

- `BehaviorSubject` **always** has a "current value" — it cannot be empty, which is why the constructor demands one. `.getValue()` throws if the subject has errored, and always returns synchronously.
- `ReplaySubject` with `bufferSize = 1` looks similar to `BehaviorSubject` but differs in one crucial way: `BehaviorSubject` **requires an initial value up front and exposes it synchronously via `.value`**; `ReplaySubject(1)` **has no value until something is emitted, and has no synchronous getter**.
- `AsyncSubject` is the odd one out: it **suppresses all intermediate emissions** and only ever emits the *last* value, and only *after* `complete()` is called. This mirrors how a `Promise` resolves once, with its final value — in fact, when you convert an Observable to a Promise via `firstValueFrom / lastValueFrom` or `toPromise()`, the semantics resemble an `AsyncSubject`.

2.4 Subject vs Observable — subscription independence

Subscribing to a plain `Subject` twice does **not** re-run any producer logic — there is no producer function, only a list of registered observers that `.next()` iterates over and calls synchronously. This is why multiple subscribers to the same `Subject` always see identical values at identical times (true multicast), whereas a cold `Observable` re-executes per subscriber.

2.5 Subjects and Angular's Change Detection

Calling `.next()` on a Subject from outside Angular's zone (e.g., inside a raw `addEventListener` or a WebSocket callback registered before `NgZone.run()`) will not trigger change detection even though subscribers fire. This matters for cross-component communication: if a service's Subject is fed from an untracked async source, template bindings depending on its subscription may appear "stuck" until the next unrelated CD cycle. Wrapping the `.next()` call in `NgZone.run()` (or using signals downstream, which Angular tracks explicitly) avoids this class of bug.

3. Code Examples

3.1 Shared-state service using `BehaviorSubject`

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

export interface CartItem {
  id: string;
  name: string;
  price: number;
  qty: number;
}

@Injectable({ providedIn: 'root' })
export class CartService {
  // Private, mutable source of truth – always has a current value.
  private readonly cartSubject = new BehaviorSubject<CartItem[]>([]);

  // Public, read-only stream – consumers cannot call next()/error()/complete().
  readonly cart$: Observable<CartItem[]> = this.cartSubject.asObservable();

  get snapshot(): CartItem[] {
    return this.cartSubject.getValue(); // synchronous read of "now"
  }

  addItem(item: CartItem): void {
    const current = this.cartSubject.getValue();
    const existing = current.find(i => i.id === item.id);

    const next = existing
      ? current.map(i => (i.id === item.id ? { ...i, qty: i.qty + item.qty } : i))
      : [...current, item];

    this.cartSubject.next(next);
  }

  removeItem(id: string): void {
    this.cartSubject.next(this.cartSubject.getValue().filter(i => i.id !== id));
  }
}
```

```

clear(): void {
  this.cartSubject.next([]);
}
}

```

```

// Consumer component – subscribes and immediately gets the CURRENT cart,
// even though CartService may have been populated long before this component existed.
@Component({
  selector: 'app-cart-badge',
  template: `<span class="badge">{{ (cartService.cart$ | async)?.length ?? 0 }}</span>`,
})
export class CartBadgeComponent {
  constructor(public cartService: CartService) {}
}

```

3.2 Notification bus using Subject

```

import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';

export interface Notification {
  type: 'success' | 'error' | 'info';
  message: string;
}

@Injectable({ providedIn: 'root' })
export class NotificationBusService {
  private readonly notifySubject = new Subject<Notification>();

  // No replay: a toast component that mounts AFTER the emission missed it –
  // and that is exactly the desired behavior for one-shot events.
  readonly notification$ = this.notifySubject.asObservable();

  success(message: string): void {
    this.notifySubject.next({ type: 'success', message });
  }

  error(message: string): void {
    this.notifySubject.next({ type: 'error', message });
  }
}

```

```

@Component({ selector: 'app-toast-host', template: `<div *ngFor="let n of active">{{
n.message }}</div>` })
export class ToastHostComponent implements OnInit, OnDestroy {
  active: Notification[] = [];
  private destroy$ = new Subject<void>();

  constructor(private bus: NotificationBusService) {}

  ngOnInit(): void {

```

```

    this.bus.notification$
      .pipe(takeUntil(this.destroy$))
      .subscribe(n => {
        this.active.push(n);
        setTimeout(() => (this.active = this.active.filter(x => x !== n)), 3000);
      });
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

```

3.3 ReplaySubject caching pattern

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, ReplaySubject, tap } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ConfigService {
  // Cache the last emission indefinitely; late subscribers (e.g. lazy-loaded
  // feature modules that inject this service after the initial load) get it instantly.
  private readonly configSubject = new ReplaySubject<AppConfig>(1);
  private loaded = false;

  readonly config$: Observable<AppConfig> = this.configSubject.asObservable();

  constructor(private http: HttpClient) {}

  load(): void {
    if (this.loaded) return;
    this.loaded = true;
    this.http.get<AppConfig>('/api/config')
      .pipe(tap(cfg => this.configSubject.next(cfg)))
      .subscribe();
  }
}

```

```

// A "recent activity log" that keeps the last 5 events for anyone subscribing later.
@Injectable({ providedIn: 'root' })
export class ActivityLogService {
  private readonly logSubject = new ReplaySubject<string>(5); // bufferSize = 5
  readonly log$ = this.logSubject.asObservable();

  record(entry: string): void {
    this.logSubject.next(`${new Date().toISOString()} - ${entry}`);
  }
}

```

3.4 toSignal() conversion

```
import { Component, computed } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { CartService } from '../cart.service';

@Component({
  selector: 'app-cart-summary',
  template: `
    <p>Items: {{ itemCount() }}</p>
    <p>Total: {{ total() | currency }}</p>
  `,
})
export class CartSummaryComponent {
  // BehaviorSubject-backed cart$ always has a current value, so no
  // initialValue option is required – toSignal picks it up synchronously.
  private cart = toSignal(this.cartService.cart$, { initialValue: [] as CartItem[] });

  itemCount = computed(() => this.cart().reduce((n, i) => n + i.qty, 0));
  total = computed(() => this.cart().reduce((sum, i) => sum + i.price * i.qty, 0));

  constructor(private cartService: CartService) {}
}
```

```
// Plain Subject (no synchronous initial value) – toSignal REQUIRES a strategy:
export class NotificationBadgeComponent {
  // Option A: explicit initialValue
  latest = toSignal(this.bus.notification$, { initialValue: null as Notification | null });

  // Option B: requiresSync – throws at runtime if the Subject hasn't emitted synchronously
  // by the time the signal is read; use only when you can guarantee an immediate emission.
  // latest = toSignal(this.bus.notification$, { requiresSync: true });

  constructor(private bus: NotificationBusService) {}
}
```

`toSignal()` automatically subscribes on creation and unsubscribes when the injection context (component/directive) is destroyed — it internally uses `DestroyRef`, so you do not need `takeUntil(destroy$)` for this specific subscription.

4. Internal Working

4.1 Subject's dual-interface implementation

RxJS implements `Subject<T>` as a class that **extends** `Observable<T>` and internally implements the Observer contract:

```
class Subject<T> extends Observable<T> {
  observers: Observer<T>[] = [];
```

```

closed = false;
hasError = false;
thrownError: any = null;

next(value: T): void {
  if (this.closed) return; // silently no-ops after complete/error in older versions;
                           // modern RxJS throws an ObjectUnsubscribedError-safe no-op
  for (const observer of this.observers.slice()) {
    observer.next(value); // synchronous, in registration order
  }
}

error(err: any): void {
  if (this.closed) return;
  this.hasError = true;
  this.thrownError = err;
  this.closed = true;
  for (const observer of this.observers.slice()) {
    observer.error(err);
  }
  this.observers.length = 0;
}

complete(): void {
  if (this.closed) return;
  this.closed = true;
  for (const observer of this.observers.slice()) {
    observer.complete();
  }
  this.observers.length = 0;
}

subscribe(observerOrNext): Subscription {
  // Overrides Observable.subscribe: instead of invoking a producer function,
  // it simply pushes the new observer into this.observers and returns a
  // Subscription whose teardown removes it from the array.
  this.observers.push(observer);
  return new Subscription(() => {
    const i = this.observers.indexOf(observer);
    if (i >= 0) this.observers.splice(i, 1);
  });
}
}

```

The critical architectural point: `Observable.subscribe()` normally invokes the subscriber function passed to `new Observable(fn)` — that's what makes plain Observables unicast/cold. `Subject` overrides `subscribe()` so that instead of running a producer function per subscriber, it just registers the observer into a shared `observers` array. `.next()` then iterates that single array and calls every registered observer synchronously. This is the entire multicast mechanism — one array, fanned out to N listeners, no re-execution.

4.2 How `BehaviorSubject` stores and replays its value

`BehaviorSubject` extends `Subject` and adds exactly one piece of state: `_value`.

```
class BehaviorSubject<T> extends Subject<T> {
  constructor(private _value: T) {
    super();
  }

  get value(): T {
    return this.getValue();
  }

  getValue(): T {
    if (this.hasError) throw this.thrownError;
    if (this.closed) throw new ObjectUnsubscribedError();
    return this._value;
  }

  next(value: T): void {
    super.next((this._value = value)); // update stored value THEN broadcast
  }

  subscribe(observer): Subscription {
    const subscription = super.subscribe(observer);
    // Immediately replay the current value to this new subscriber only,
    // synchronously, before subscribe() returns.
    if (!subscription.closed) {
      observer.next(this._value);
    }
    return subscription;
  }
}
```

So the "replay" isn't a buffer of history — it's a single overridden `subscribe()` that fires `observer.next(this._value)` once, immediately upon subscription, in addition to registering the observer for future broadcasts. That's why late subscribers "catch up" to exactly one value: the most recent.

4.3 How `ReplaySubject` replays multiple values

`ReplaySubject` maintains an actual buffer (array) of past emissions, optionally paired with timestamps:

```
class ReplaySubject<T> extends Subject<T> {
  private buffer: (T | number)[] = []; // interleaved values (and timestamps if windowTime set)

  constructor(
    private bufferSize = Infinity,
    private windowTime = Infinity,
  ) {
    super();
  }
}
```

```

}

next(value: T): void {
  if (!this.closed) {
    this.buffer.push(value);
    this.buffer.length > this.bufferSize && this.buffer.shift(); // trim to bufferSize
    // if windowTime is finite, also trim entries older than windowTime
  }
  super.next(value);
}

subscribe(observer): Subscription {
  const subscription = super.subscribe(observer);
  for (const value of this._trimmedBufferSnapshot()) {
    if (!subscription.closed) observer.next(value);
  }
  return subscription;
}
}

```

Every new subscriber gets the entire (trimmed) buffer replayed synchronously, in original order, before receiving any future live emissions — hence "Replay."

4.4 How AsyncSubject withholds emissions

```

class AsyncSubject<T> extends Subject<T> {
  private value: T | null = null;
  private hasNext = false;

  next(value: T): void {
    if (!this.closed) {
      this.value = value;
      this.hasNext = true; // just remember it; do NOT broadcast yet
    }
  }

  complete(): void {
    if (!this.closed && this.hasNext) {
      // Only NOW broadcast the single last value, then complete.
      super.next(this.value!);
    }
    super.complete();
  }
}

```

This is why `AsyncSubject` is the RxJS analog of a resolved Promise: intermediate `.next()` calls are silently swallowed until `.complete()` finally releases the last one.

5. Edge Cases & Gotchas

1. **BehaviorSubject demands an initial value at construction time.** `new BehaviorSubject<User | null>(null)` is the idiomatic way to model "no user yet" — you cannot construct one with "no value" the way you can a plain `Subject`. Forgetting this and reaching for `Subject` instead when you actually need synchronous "current state" access is a very common mistake that breaks `*ngIf="user$ | async as user"` patterns relying on an immediate emission.
2. **ReplaySubject buffer memory growth.** `new ReplaySubject<T>()` with no arguments defaults `bufferSize` to `Infinity` — it will retain *every* value ever emitted, for the lifetime of the Subject. In a long-lived singleton service (e.g., a WebSocket message log), this is a slow, silent memory leak. Always pass an explicit bounded `bufferSize` (and consider `windowTime` for time-based eviction) unless you specifically want unbounded history.
3. **Calling `.next()` after `.complete()` (or `.error()`) is a silent no-op.** Once a Subject is `closed`, further `.next()` calls do nothing — no error is thrown, no observers are notified, and there's no console warning. This produces confusing bugs where a service appears to "stop working" because something upstream called `.complete()` prematurely (a very common bug: calling `destroy$.complete()` and then later still trying to `destroy$.next()` on a service that outlives its expected scope).
4. **Subscribing after `.complete()` still works differently per type**, and this trips people up:
 - Plain `Subject`: a subscriber added after `complete()` immediately receives `complete()` and nothing else (no error, but no values either).
 - `BehaviorSubject` / `ReplaySubject`: even after completion, new subscribers still receive the replayed value(s) *first*, then `complete()`.
 - `AsyncSubject`: a subscriber added after completion receives the final buffered value, then `complete()` — this is actually the "correct"/expected use case.
5. **Exposing raw Subjects vs. `asObservable()`.** If a service exposes `readonly cart$ = this.cartSubject` directly (instead of `.asObservable()`), any consumer can cast/access it and call `.next()`, `.error()`, or `.complete()` from outside the service, breaking encapsulation and the unidirectional data flow the store pattern relies on. `asObservable()` returns a thin `Observable` wrapper that forwards subscriptions but hides the Observer methods from the public type — always expose `.asObservable()` (or, in TypeScript, at minimum type the public field as `Observable<T>` even if you skip the wrapper — but `asObservable()` is the safer, idiomatic choice since it also prevents a runtime-savvy caller from doing `(service.cart$ as Subject<T>).next(...)`).
6. **`getValue()` throws if the BehaviorSubject has errored or is closed.** Reading `.value` synchronously after an unhandled `.error()` call will throw, not return `undefined` — guard with `try/catch` or check `hasError`-adjacent application state if this is reachable.
7. **Multiple `.next()` calls in the same synchronous tick with `combineLatest` / multi-Subject state stores can cause "glitches"** — intermediate, inconsistent combined states — because each Subject broadcasts synchronously and independently the instant `.next()` is called, rather than batching. This is one motivator for moving shared state to Angular `signal()` / `computed()` (which batch and de-duplicate via glitch-free propagation) instead of raw multi-Subject stores.
8. **`toSignal()` without `initialValue` on a Subject with no synchronous first emission throws at runtime** (in strict mode) or produces `undefined`-typed signals — you must supply `initialValue` (safe

default) or `requiresSync: true` (only safe if you can *guarantee* — e.g., because the source is a `BehaviorSubject` or `ReplaySubject(1)` already primed — a synchronous emission on subscribe).

9. **Unsubscribing/memory leaks are still your responsibility for manual `.subscribe()` calls on Subjects** — Subjects don't complete on their own just because a component was destroyed; the classic `destroy$` pattern (or `takeUntilDestroyed()` in modern Angular) is still required, `toSignal()`'s automatic cleanup only covers the specific subscription it creates internally.

6. Interview Questions & Answers

Q1. What is a Subject in RxJS, and how is it different from a plain Observable?

A `Subject` is both an `observable` (you can subscribe to it) and an `observer` (it has `.next()`, `.error()`, `.complete()`), which lets external code push values into it imperatively. A plain `Observable` is unicast and typically cold — each subscription re-runs the producer independently — while a `Subject` is multicast: all subscribers share one execution and receive the same values at the same time.

Q2. Why does `BehaviorSubject` require an initial value in its constructor, but `Subject` does not?

Interviewer intent: checks whether the candidate understands `BehaviorSubject`'s core contract — "there is always a current value" — versus just knowing it as "a `Subject` with a default."

`BehaviorSubject` guarantees that any subscriber, no matter when it subscribes, immediately and synchronously receives a value (either the seeded initial value or the most recent `.next()` call). Since that guarantee must hold even before any `.next()` has ever been called, the type system and the class enforce providing a seed value at construction. `Subject` makes no such guarantee — subscribers only see values emitted *after* they subscribe — so it has nothing to seed.

Q3. What's the practical difference between `ReplaySubject(1)` and `BehaviorSubject`?

They look similar (both replay one value to late subscribers) but differ in two ways: (1) `BehaviorSubject` requires and exposes a synchronous "current value" via `.value` / `.getValue()` even before any subscription happens; `ReplaySubject(1)` has no such synchronous getter and holds no value until the first `.next()`. (2) `ReplaySubject` can be configured with a `windowTime` for time-based buffer expiry and an arbitrary `bufferSize`, capabilities `BehaviorSubject` doesn't have (it's always exactly buffer-size 1, with no time expiry).

Q4. Explain what "multicast" means in RxJS and how a Subject achieves it internally.

Multicast means one producer execution is shared across many consumers, as opposed to unicast where each consumer gets its own independent execution. Internally, `Subject` overrides `Observable.subscribe()`: rather than invoking a producer function per subscriber (which is what makes plain `Observables` cold), it simply appends the new observer to an internal `observers` array. `.next(value)` then iterates that single array synchronously and calls `.next(value)` on every registered observer. There's exactly one source of truth for "what was emitted and when" — that array — which is the entire multicast mechanism.

Q5. When would you choose an `AsyncSubject` over a resolved `Promise` or `firstValueFrom`?

Interviewer intent: tests whether the candidate has actually used `AsyncSubject` or is just reciting the table — a fair signal for depth, since this type is genuinely rare in app code.

In practice, almost never in application code — `AsyncSubject` is mostly of academic/library interest, since it only emits its last value once `.complete()` fires, mirroring `Promise` semantics almost exactly. You'd reach for it only if you specifically needed `Observable`-chain composability (operators, cancellation via `unsubscribe`, multiple subscribers observing the same eventual result) around a "run once, get final value" process, rather

than converting to a Promise outright. In modern RxJS, `firstValueFrom` / `lastValueFrom` typically replace both `AsyncSubject` and the deprecated `.toPromise()`.

Q6. What happens if you call `.next()` on a Subject after calling `.complete()`?

Nothing observable happens — it's a silent no-op. Once `closed` is set to `true` internally (by either `.complete()` or `.error()`), subsequent `.next()` calls return immediately without notifying any observers and without throwing. This is a common source of "silently dead" services when `.complete()` is called too early or on a singleton that's expected to live longer.

Q7. Why should a service expose `subject.asObservable()` rather than the raw Subject?

Exposing the raw Subject leaks Observer capabilities (`.next()`, `.error()`, `.complete()`) to any consumer, letting arbitrary code outside the service mutate shared state or terminate the stream — breaking encapsulation and making state changes hard to trace. `asObservable()` returns a thin wrapper exposing only the `observable` surface, enforcing that state can only be mutated through the service's own public methods (a unidirectional data flow, similar in spirit to how NgRx enforces mutation only through dispatched actions).

Q8. How would you implement a cross-component communication mechanism between two sibling components that don't share a parent-child relationship?

Create a shared service (Angular's DI makes singletons trivial) holding a `Subject` (for one-off events) or a `BehaviorSubject` (for shared state with a "current value"). Sibling A injects the service and calls `.next()` / a wrapper method to emit; Sibling B injects the same service instance and subscribes to the exposed `.asObservable()` stream. Because both components resolve the same provided instance from the injector, this sidesteps `@Input()` / `@Output()` entirely and works regardless of tree position.

Q9. What's the danger of using an unbounded `ReplaySubject` as a long-lived application-wide event log?

`new ReplaySubject<T>()` defaults `bufferSize` to `Infinity`, so it retains every emitted value for the lifetime of the Subject. In a long-lived singleton (e.g., root-provided service), this buffer grows without bound as the app runs, causing a slow memory leak and, for large payloads, real performance degradation on every new subscription (since the entire buffer gets replayed). The fix is always passing an explicit, bounded `bufferSize` (and/or `windowTime`) appropriate to the use case.

Q10. How do you convert a `BehaviorSubject`-backed observable to a signal, and what's different versus converting a plain `Subject`?

Use `toSignal(source$)`. For a `BehaviorSubject`-backed stream, `toSignal` can synchronously read the initial value on subscription (since `BehaviorSubject` always emits immediately), so it doesn't strictly need an `initialValue` option — though providing one keeps the resulting signal's type non-nullable/well-defined. For a plain `Subject`, there is no guaranteed synchronous first emission, so `toSignal` requires you to either supply `initialValue` (used until the first real emission arrives) or pass `requiresync: true` and accept a runtime throw if no synchronous emission occurs.

Q11. Multiple components share a `BehaviorSubject`-backed `AuthService.currentUser$`. One component calls `.next(null)` directly on the subject instead of a `logout()` method. What's wrong with this design, and how do you prevent it?

Interviewer intent: probing for architecture/encapsulation awareness beyond rote RxJS trivia — a strong signal of production experience.

The problem is that the Subject is exposed in a way that lets any consumer mutate the "single source of truth" for auth state directly, bypassing any side effects the service's `logout()` method is supposed to perform (clearing tokens, calling a logout endpoint, redirecting, clearing other caches). This makes state changes untraceable and easy to corrupt. The fix: keep the `BehaviorSubject` `private`, expose only `currentUser$ = this.userSubject.asObservable()`, and require all mutation to go through explicit public methods (`login()`, `logout()`) on the service — even within the same file, never destructure or cast to get at the private Subject.

Q12. Why can synchronous cascading `.next()` calls across multiple related Subjects cause a "glitch," and how does moving to Angular signals address it?

If two Subjects hold logically related pieces of state (e.g., `firstName$` and `fullName$` derived by combining `firstName$ / lastName$` via `combineLatest()`), updating one triggers an immediate, synchronous re-emission of the combined stream — but if you update both source Subjects in sequence, subscribers can observe a transient, inconsistent intermediate combination between the two `.next()` calls (a "glitch"). Angular's `signal() / computed()` avoid this because computed signals are lazily and consistently recomputed via a dependency graph with automatic de-duplication — a computed signal is guaranteed never to be read in an intermediate/inconsistent state, since recomputation happens on read, after all upstream writes in the current synchronous block have settled.

Q13. What does `share()` or `shareReplay()` have to do with Subjects?

Both operators multicast a source Observable by internally routing its emissions through a Subject (`share()` typically wraps a plain `Subject`; `shareReplay()` wraps a `ReplaySubject`), converting what would otherwise be independent cold subscriptions into a single shared execution with fan-out — exactly the mechanism a hand-rolled Subject-based service provides, but composed declaratively as a pipeable operator rather than an explicit class field.

Q14. If a component subscribes to a `ReplaySubject(3)` after 5 values have already been emitted, what does it receive, and in what order?

It receives the last 3 buffered values (the most recent ones, oldest of the three first), replayed synchronously in original emission order, immediately upon subscription — followed by any subsequent live emissions. It does not receive the first 2 of the 5 already-evicted values, since the buffer only ever retains the most recent `bufferSize` entries.

7. Quick Revision Cheat Sheet

- **Subject** = Observable + Observer in one object → multicast, no replay, no initial value.
- **BehaviorSubject** = Subject + mandatory initial value + synchronous `.value / .getValue()` + replays exactly 1 (the current) value to late subscribers.
- **ReplaySubject(n, windowTime?)** = Subject + buffer of up to `n` values (optionally time-bounded) replayed in order to late subscribers; default `bufferSize` is `Infinity` — always bound it.
- **AsyncSubject** = Subject that swallows all `.next()` calls until `.complete()`, then emits only the last value — Promise-like, rarely used directly.
- Multicast mechanism = `subject.subscribe()` overrides `observable.subscribe()` to register into a shared `observers` array instead of re-invoking a producer function; `.next()` iterates that array synchronously.
- `.next()` after `.complete() / .error()` → silent no-op, no error thrown.
- Always expose `subject.asObservable()` publicly; keep the mutable Subject `private`.

- Cross-component communication pattern: shared `@Injectable({ providedIn: 'root' })` service + `Subject/BehaviorSubject` + `asObservable()`.
- `toSignal(source$)` needs `initialValue` (or `requiresSync: true`) unless the source guarantees a synchronous emission on subscribe (true for `BehaviorSubject`/primed `ReplaySubject`, not for plain `Subject`).
- `toSignal()` auto-unsubscribes via `DestroyRef` on injector/component destroy — but only for that one subscription; manual `.subscribe()` calls still need `takeUntilDestroyed()` / `destroy$`.
- `share()` / `shareReplay()` are Subject-based multicasting operators wrapped declaratively.

Created By - Durgesh Singh